13601-02, 13601-03: 알고리즘

Spring 2026, School of Computer Science and Engineering

Practice 2 – 탐욕적 알고리즘

제출자: 최세현(20235685)

* 이 템플릿은 과제 수행 참고를 위한 예시 템플릿이며 자유롭게 템플릿을 변형해 사용하여도 무방합니다.

코드 – 프림(배열/우선순위 큐) & 크루스칼 알고리즘

```
import time
import heapq
import numpy as np

# 데이터 파싱 함수
def read_graph_data(filename):
    with open(filename, 'r') as f:
        lines = f.readlines()
        V, E = map(int, lines[0].split())
        edges = []
        for line in lines[1:]:
            u, v, w = map(int, line.split())
            edges.append((u, v, w))
    return V, E, edges

# ==========================================
# 1. 프림 알고리즘: 인접 행렬 + 배열 (Numpy 최적화)
# ==========================================
def prim_matrix_numpy(V, edges):
    # 인접 행렬 생성 및 초기화 (무한대)
    adj_matrix = np.full((V, V), np.inf)
    for u, v, w in edges:
        adj_matrix[u][v] = w
        adj_matrix[v][u] = w # 무방향 그래프

    visited = np.zeros(V, dtype=bool)
    min_weight = np.full(V, np.inf)
    min_weight[0] = 0 # 시작 정점(0번) 가중치는 0

    mst_weight_sum = 0

    start_time = time.perf_counter()

    for _ in range(V):
        # 1. np.where와 np.argmin으로 방문 안 한 정점 중 최소 가중치 정점 찾기
        unvisited_weights = np.where(visited, np.inf, min_weight)
        u = int(np.argmin(unvisited_weights))

        # 방문 처리 및 가중치 합산
        visited[u] = True
        mst_weight_sum += min_weight[u]

        # 2. np.minimum으로 반복문 없이 인접 정점의 최소 가중치 배열 일괄 갱신
        min_weight = np.minimum(min_weight, adj_matrix[u])

    end_time = time.perf_counter()
```

```
    return int(mst_weight_sum), end_time - start_time

# ==========================================
# 2. 프림 알고리즘: 인접 리스트 + 우선순위 큐 (Min-heap)
# ==========================================
def prim_list_heap(V, edges):
    # 인접 리스트 구성
    adj_list = {i: [] for i in range(V)}
    for u, v, w in edges:
        adj_list[u].append((w, v))
        adj_list[v].append((w, u))

    visited = [False] * V
    mst_weight_sum = 0

    # (가중치, 정점) 형태의 힙
    min_heap = [(0, 0)]

    start_time = time.perf_counter()

    while min_heap:
        weight, u = heapq.heappop(min_heap)

        if visited[u]:
            continue

        visited[u] = True
        mst_weight_sum += weight

        for next_weight, v in adj_list[u]:
            if not visited[v]:
                heapq.heappush(min_heap, (next_weight, v))

    end_time = time.perf_counter()
    return mst_weight_sum, end_time - start_time

# ==========================================
# 3. 크루스칼 알고리즘: 간선 리스트 + Union-Find
# ==========================================
# Union-Find 자료구조 (경로 압축 적용)
def find(parent, i):
    if parent[i] == i:
        return i
    parent[i] = find(parent, parent[i]) # 경로 압축
    return parent[i]

def union(parent, rank, x, y):
    xroot = find(parent, x)
    yroot = find(parent, y)

    if rank[xroot] < rank[yroot]:
        parent[xroot] = yroot
    elif rank[xroot] > rank[yroot]:
        parent[yroot] = xroot
    else:
        parent[yroot] = xroot
        rank[xroot] += 1

def kruskal_uf(V, edges):
    parent = [i for i in range(V)]
    rank = [0] * V
    mst_weight_sum = 0
```

```
    edges_count = 0

    start_time = time.perf_counter()

    # 간선을 가중치 기준으로 오름차순 정렬
    sorted_edges = sorted(edges, key=lambda item: item[2])

    for u, v, w in sorted_edges:
        x = find(parent, u)
        y = find(parent, v)

        # 사이클이 생기지 않는다면 간선 추가
        if x != y:
            mst_weight_sum += w
            union(parent, rank, x, y)
            edges_count += 1
            if edges_count == V - 1: # MST 완성 조건
                break

    end_time = time.perf_counter()
    return mst_weight_sum, end_time - start_time

# ==========================================
# 메인 실행부
# ==========================================
def run_and_evaluate(filename, graph_type):
    print(f"\n[{graph_type} ({filename}) 분석 결과]")
    try:
        V, E, edges = read_graph_data(filename)
    except FileNotFoundError:
        print(f"오류: '{filename}' 파일이 같은 폴더에 없습니다. 파일을 준비해주세요.")
        return

    # 1. 프림 (행렬+배열)
    w1, t1 = prim_matrix_numpy(V, edges)
    print(f"1. 프림 (행렬+배열)   | 가중치: {w1} | 소요 시간: {t1:.6f} 초")

    # 2. 프림 (리스트+힙)
    w2, t2 = prim_list_heap(V, edges)
    print(f"2. 프림 (리스트+힙)   | 가중치: {w2} | 소요 시간: {t2:.6f} 초")

    # 3. 크루스칼 (간선리스트+UF)
    w3, t3 = kruskal_uf(V, edges)
    print(f"3. 크루스칼 (간선+UF) | 가중치: {w3} | 소요 시간: {t3:.6f} 초")

    if w1 == w2 == w3:
        print("=> 검증 완료: 세 알고리즘의 MST 가중치가 동일합니다.")
    else:
        print("=> 경고: 가중치 결과가 다릅니다. 데이터를 확인하세요.")

if __name__ == "__main__":
    # 파일명은 실제 제공된 파일명과 동일하게 맞춰주세요.
    run_and_evaluate("sparse_graph.txt", "희소 그래프")
    run_and_evaluate("dense_graph.txt", "밀집 그래프")
```

1번 코드 실행 결과

```
sehn00@sehn-pc:~/algorithm_4_1/hw2
$ python3 mst.py

[희소 그래프 (sparse_graph.txt) 분석 결과]
1. 프림 (행렬+배열)   | 가중치: 152752 | 소요 시간: 0.055924 초
2. 프림 (리스트+힙)   | 가중치: 152752 | 소요 시간: 0.015765 초
3. 크루스칼 (간선+UF) | 가중치: 152752 | 소요 시간: 0.004947 초
=> 검증 완료: 세 알고리즘의 MST 가중치가 동일합니다.

[밀집 그래프 (dense_graph.txt) 분석 결과]
1. 프림 (행렬+배열)   | 가중치: 4999 | 소요 시간: 0.047088 초
2. 프림 (리스트+힙)   | 가중치: 4999 | 소요 시간: 25.874076 초
3. 크루스칼 (간선+UF) | 가중치: 4999 | 소요 시간: 0.747763 초
=> 검증 완료: 세 알고리즘의 MST 가중치가 동일합니다.
sehn00@sehn-pc:~/algorithm_4_1/hw2
$
```

결과 분석 및 고찰

(1) 희소 그래프 (Sparse Graph)에서의 자료구조별 성능 차이

간선의 수가 정점의 수에 비해 현저히 적은 희소 그래프 환경에서는 간선 위주로 접근하는 알고리즘들이 압도적인 성능 우위를 보였다. 간선 리스트를 가중치 기준으로 정렬한 뒤 탐색하는 크루스칼 알고리즘은 0.0049초를 기록하며 가장 빠른 속도를 보여주었다. 인접 리스트와 우선순위 큐(Min-heap)를 결합한 프림 알고리즘 역시 0.0157초로 매우 효율적으로 동작했다. 이 두 방식은 그래프가 희소할수록 메모리 공간 낭비가 적고, 탐색해야 할 간선의 절대적인 개수(E) 자체가 적기 때문에 정렬이나 힙 재구조화에 드는 오버헤드가 최소화되었다.

반면, 인접 행렬을 기반으로 한 프림 알고리즘은 0.0559초가 소요되어 상대적으로 가장 비효율적이었다. 이는 인접 행렬 구조의 태생적 한계인 O(V^2)의 시간 복잡도 때문이다. 정점 간의 실제 연결 여부나 가중치 유무와 상관없이, 매 턴마다 V개의 모든 배열 요소를 순회하며 최소값을 찾아야 한다. 특히 간선이 듬성듬성 존재하는 희소 그래프에서는 무한대(INF) 값만 들어있는 배열 공간을 무의미하게 반복 조회하게 되며, 이로 인해 막대한 CPU 사이클 낭비와 메모리 대역폭 손실이 발생했음을 알 수 있다.

(2) 밀집 그래프 (Dense Graph)에서의 성능 역전과 인터프리터 한계 극복

정점 간의 연결이 매우 빽빽하여 간선의 수가 최대치(V^2)에 근접하는 밀집 그래프에서는 기존의 상식을 뒤엎는 결과가 도출되었다. 희소 그래프에서 우수했던 인접 리스트 기반 프림 알고리즘은 무려 25.87초라는 최악의 수행 시간을 기록했다. 이는 밀집 그래프 특성상 특정 정점을 방문할 때마다 수백, 수천 개의 인접 간선이 한꺼번에 우선순위 큐(Heap)에 쏟아져 들어오기 때문이다. 파이썬 환경에서 이토록 거대한 트리 구조를 유지하고 지속적으로 데이터의 삽입 및 삭제(Heapify)를 수행하는 것은 잦은 동적 메모리 할당과 포인터 추적(Pointer Chasing)을 유발하여 치명적인 성능 저하로 이어졌다. 크루스칼 알고리즘 역시 폭발적으로 증가한 간선 전체를 우선 정렬(Sorting)해야 하는 막대한 초기 오버헤드 때문에 0.74초로 속도가 저하되었다.

반면, 희소 그래프에서 가장 느렸던 인접 행렬 기반 프림 알고리즘은 밀집 그래프에서 0.047초라는 압도적인 속도로 1위를 차지했다. 여기서 주목할 점은 순수 파이썬의 반복문(for-loop)이 아닌, Numpy 라이브러리의 벡터화(Vectorization) 연산을 적극 활용했다는 것이다. 파이썬은 인터프리터 언어의 특성상 객체 타입 검사와 GIL(Global Interpreter Lock)로 인해 단순 반복 연산에 심각한 병목이 존재한다. 하지만 Numpy의 where, argmin, minimum 메서드를 활용함으로써, 이러한 반복 연산을 파이썬 레벨이 아닌 하위 C 언어 레벨로 끌어내려 일괄 처리할 수 있었다. 연속된 메모리 블록(Array)을 다루는 인접 행렬의 특성과 Numpy의 벡터화가 시너지를 일으켜, 밀집 그래프의 방대한 데이터를 병목 없이 단숨에 처리해 낸 것이다.

(3) 종합 결론

본 과제의 결과는 알고리즘을 설계하고 구현할 때 단순히 점근적 표기법(Big-O) 형태의 이론적 시간 복잡도에만 의존해서는 안 되며, 실제 주어지는 입력 데이터의 특성(밀집도)과 이를 구동하는 프로그래밍 언어의 내부 동작 방식까지 종합적으로 분석해야 함을 시사한다.

일반적인 이론에 따르면 간선 중심의 탐색이 효율적이라고 알려져 있으나, 이는 데이터가 희소(Sparse)할 때만 성립한다. 간선이 적은 환경에서는 크루스칼(O(E log E))이나 우선순위 큐 기반의 프림 알고리즘(O(E log V))이 불필요한 정점 탐색을 줄여주어 유리했다. 그러나 정점 간 연결이 포화 상태에 이르는 밀집 그래프에서는 상황이 완전히 역전되었다. 막대한 수의 간선을 모두 정렬하거나 힙 트리를 지속적으로 삽입 및 삭제(Heapify)하는 과정에서 발생하는 데이터 이동 오버헤드가 오히려 알고리즘의 성능을 크게 저하시켰기 때문이다.

결과적으로 간선이 폭발적으로 증가하는 밀집 네트워크 환경에서는 인접 행렬 기반(O(V^2))의 탐색 방식이 간선의 개수 변화에 영향을 받지 않고 일정한 탐색 횟수를 보장하여 안정적이었다. 특히 파이썬과 같은 인터프리터 언어 환경에서는 내장 반복문을 사용할 때 막대한 속도 저하가 발생하는데, 행렬 구조를 채택한 덕분에 Numpy의 벡터화(Vectorization) 메서드를 적용할 수 있었다. 이를 통해 병목이 발생하는 반복 연산을 하위 C 언어 레벨에서 일괄 처리함으로써 수행 속도를 단축시킬 수 있었으며, 최적의 문제 해결은 알맞은 자료구조의 선택과 언어적 특성을 고려한 코드 최적화가 맞물릴 때 비로소 완성됨을 확인했다.